

Stellar no projeto Holdfy

Documento de referência — gerado em 31/05/2026

Resumo executivo

No monorepo Holdfy, a **Stellar** aparece exclusivamente no stack **APICash** (*money/apicash*). Gatebox, o simulador de banco (app_banco) e a auditoria blockchain do admin (Polygon) **não** usam Stellar.

Em uma frase: **PIX é o trilho fiat; Stellar/Soroban é o trilho de custódia e liquidação tokenizada do escrow** (ativo BRLx na rede).

Diagrama de arquitetura

Visão de quais componentes do monorepo dependem da rede Stellar.

Figura 2 — Stellar só no stack APICash (não no Gatebox nem no banco simulador)

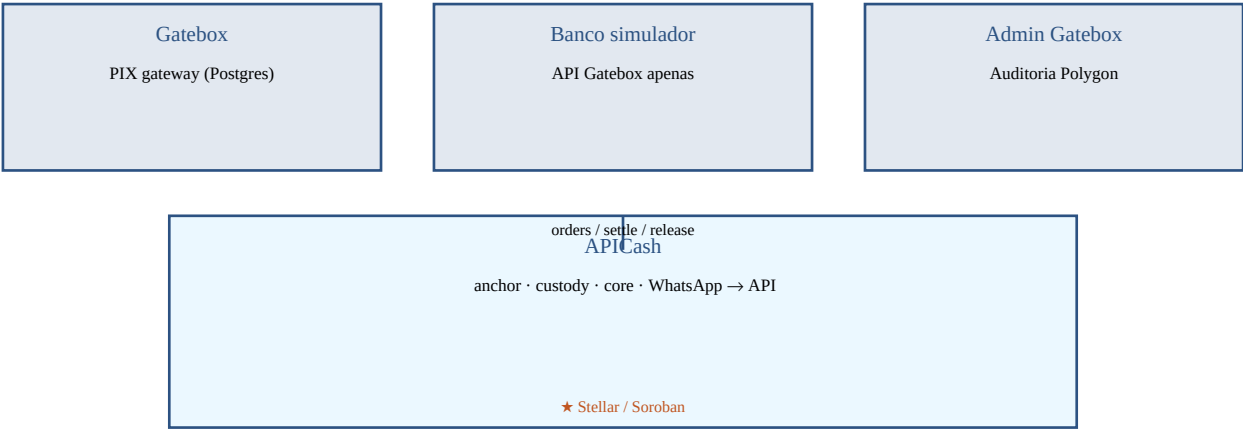
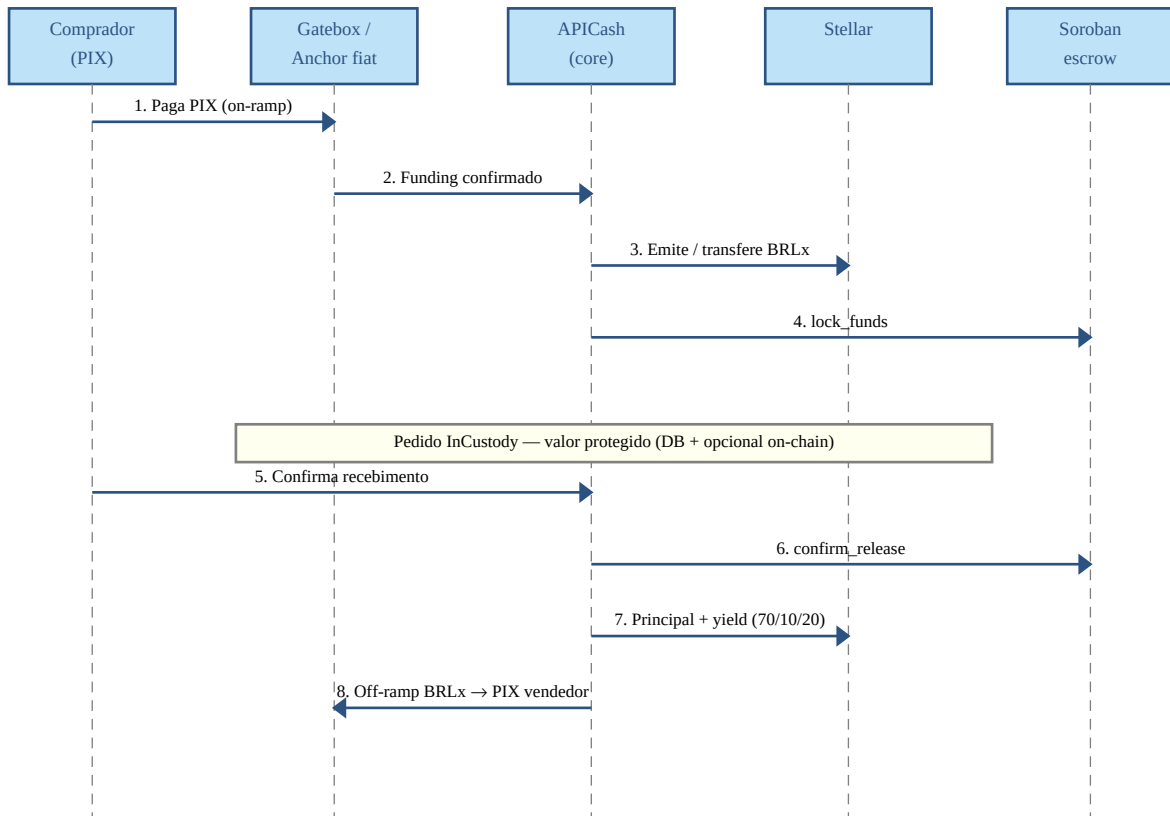


Diagrama de sequência

Equivalente ao diagrama Mermaid da documentação: do pagamento PIX até a liberação e o off-ramp ao vendedor.

Figura 1 — Fluxo de sequência APICash + Stellar/Soroban



Participantes: comprador (PIX), Gatebox/anchor fiat, APICash core, rede Stellar (BRLx) e contrato Soroban (escrow).

Onde entra e onde não entra

Área	Stellar?	Função
APICash (anchor, custody, soroban-contrato)	Sim	BRLx, on/off-ramp, escrow, yield on-chain
Gatebox (gateway PIX)	Não	PIX em Postgres; sem ledger Stellar
Banco simulador (Flutter + Go)	Não	Integra só API externa do Gatebox
WhatsApp HoldFy	Indireto	Chama apicash-core (orders, custódia, PIX)
x402 (micropagamentos)	Não	Base Sepolia (Ethereum), protocolo à parte

Fluxo de negócio (passos)

- **1. Criar pedido** — antifraude → anchor gera instrução PIX (on-ramp) → status pending_funding.
- **2. PIX pago** — settle confirma funding → transfere BRLx para escrow → lock_funds (DB + Soroban opcional).
- **3. Comprador confirma entrega** — custody/release → contrato libera → off-ramp PIX ao vendedor.

Componentes Rust e responsabilidades

apicash-anchor — ponte fiat ↔ Stellar

- On-ramp: PIX → BRLx (token na Stellar; APICASH_STELLAR_ASSET_CODE, padrão BRLx).
- Off-ramp: BRLx → PIX ao vendedor.
- Horizon (APICASH_STELLAR_HORIZON_URL) para consultar transações.
- Dev: APICASH_FIAT_RAIL=simulated sem rede real.

apicash-custody — escrow e yield

- Trava principal, calcula rendimento, libera ao vendedor.
- Split do yield: 70% vendedor / 10% comprador / 20% plataforma.
- Com APICASH_SOROBAN_ENABLED=1: invoca contrato via CLI stellar (lock/release).
- Sem Soroban: modo mock (hashes mock_*) — comum em desenvolvimento local.

soroban-contracts — smart contracts

Contratos Soroban: lock_funds, confirm_release, mark_disputed, split de yield. Wasm: target/wasm32v1-none/release/apicash_soroban_contracts.wasm. Deploy via scripts/soroban-testnet-deploy.sh.

apicash-core — orquestração

Persiste por pedido: anchor_tx_hash, brlx_escrow_transfer_tx_hash, soroban_escrow_contract_id, soroban_lock_tx_hash, soroban_mode (mock | soroban | pending_funding).

Rede e configuração

Por padrão usa **Stellar testnet pública** (não é nó local):

- Horizon: <https://horizon-testnet.stellar.org>
- Soroban RPC: <https://soroban-testnet.stellar.org>

Variáveis em money/.env.example: APICASH_SOROBAN_*, APICASH_STELLAR_*, emissor BRLx, contrato de escrow. Com APICASH_REQUIRE_TESTNET=1 a API rejeita hashes mock.

Por que Stellar (e não só Postgres)?

- Custódia auditável — escrow com hash on-chain.
- Ativo programável (BRLx) — token SEP-41 / SAC na Soroban.
- Yield on-chain — rendimento com split automático no contrato.
- Padrão Anchor (SEP-24) — ramp fiat ↔ crypto na rede Stellar.

O Gatebox continua sendo o PIX operacional; a Stellar é a camada de settlement tokenizado do APICash (ou simulada em dev).

Estado prático do projeto

Segundo PLANO_EXECUCAO.md: escrow/PIX/yield ~85% funcional; Soroban frequentemente em mock até Postgres + deploy testnet + APICASH_SOROBAN_ENABLED=1. Gatebox e banco simulador não

dependem de Stellar.

Perguntas frequentes

Pergunta	Resposta
Stellar é obrigatória para o PIX do Gatebox?	Não.
Stellar é o quê no HoldFy?	Ledger + escrow tokenizado (BRLx) do APICash.
Soroban faz o quê?	Contrato de lock/release e split de yield.
WhatsApp usa Stellar direto?	Não — usa API APICash, que pode acionar anchor/custódia.
Blockchain do admin Gatebox?	Polygon (âncora de auditoria), outro propósito.

Referências: *CLAUDE.md*, *money/.env.example*, *crates apicash-anchor*, *apicash-custody*, *soroban-contracts*, *apicash-core (order_handler)*. Diagramas gerados em vector via ReportLab (equivalentes ao Mermaid do chat).